

NAME: ADEBIMPE ADEROJU M.

COURSE: CSE210- PROGRAMMING WITH FUNCTIONS

INSTRUCTURE'S NAME: JOHN READING

Foundation Program #1: Abstraction with YouTube Videos

This program models YouTube videos for product awareness tracking, using abstraction to provide Video and Comment classes with focused public interfaces for managing and querying video data without exposing internal storage details.

Classes and Attributes/Behaviors

Video Class

- Video
 - title: string
 - author: string
 - length: int (in seconds)
 - comments: List //Comment
- Video ()
 - + set_title(string)
 - + set_author(string)
 - + set_length(int)
 - + add_comment(Comment)
 - + get_num_comments(): int

Responsibilities: Store video info, manage comments list, return comment count via public method.

Comment Class

- Comment
 - commenter: string
- text: string
 - + Comment (string name, string text)

Responsibilities: Hold commenter name and text for Video storage.

Program Execution Flow

- Instantiate 3-4 Video objects, set title/author/length via setters.
- For each Video, create 3-4 Comment objects and call add_comment().
- Add Videos to a list.
- Iterate list: Display title, author, length, get_num_comments(), then all comments.

Foundation Program #2: Encapsulation with Online Ordering

This program implements an e-commerce ordering system with encapsulation, using private fields and public methods in Product, Customer, Address, and Order classes to protect data while enabling order processing, labels, and pricing.

Classes and Attributes/Behaviors

Product Class

- Product
 - name: string
 - productId: string
 - price: double
 - quantity: int
- Product()
 - + set_name(string) / get_name(): string
 - + set_productId(string) / get_productId(): string
 - + set_price(double) / get_price(): double
 - + set_quantity(int) / get_quantity(): int
 - + get_total_cost(): double // price * quantity

Address Class

- Address
 - street: string
 - city: string
 - state: string
 - country: string
- Address()

- + setters/getters for all fields
- + is_in_NIGERIA(): bool
- + get_full_address(): string // formatted with newlines

is_in_NIGERIA() returns true if country == "NIGERIA"; get_full_address() concatenates fields.

Customer Class

- Customer
 - name: string
 - address: Address
- Customer()
 - + setters/getters
 - + lives_in_NIGERIA(): bool // calls address.is_in_NIGERIA()

Order Class

- Order
 - products: List<Product>
 - customer: Customer
 - shipping_cost: double (computed)
 - total_cost: double (computed)
- + Order()
 - + add_product(Product)
 - + calculate_total(): void
 - + get_packing_label(): string // list name + productId
 - + get_shipping_label(): string // customer name + address
 - + get_total_price(): double // sum products + shipping (\$5 NIGERIA, \$35 else)

calculate_total() sums product totals + shipping (based on customer.lives_in_NIGERIA()).

Program Execution Flow

Create 2+ Address, Customer instances; set data.

Create 2+ Order instances with customer.

For each Order, create 2-3 Product, add to Order.

Call calculate_total(), display packing_label, shipping_label, total_price.

No user input; hard-coded data and console output.